The Lock-Step Tax: Per-Lane Control Flow as the Missing Primitive in Tile GPU DSLs

Anonymous Author(s)

Abstract

Tile abstractions are how GPU kernels are written now, in Triton and increasingly in the vendor's own stack, and on dense tensor work they match hand-written CUDA. On irregular, control-driven kernels, from automata to mixture-of-experts routing, they lose an order of magnitude. Whether that gap is a tuning deficit or a property of the abstraction is a question about language design, and it has not been answered mechanistically.

We answer it. The gap is structural, and one shared loop latch explains it. A tile program's lanes advance together, so a data-dependent loop runs as long as its warp's slowest lane and a dependent load cannot be overlapped across lanes. On an NFA worklist the gap decomposes into a launch-configuration artifact, a redundancy removable by lane-packing, and an irreducible ~2× residual. At matched occupancy, issuing *fewer* warp-instructions than CUDA and below both roofline ceilings, the tile kernel still spends 15.3× more cycles stalled on a dependent load. The same mechanism predicts, and we confirm, that the residual disappears for a memory-bound DFA once its table spills to DRAM.

The missing primitive is a per-lane loop exit, and we prove TritonGPU cannot express it: `scf.condition` carries a single `i1`. We therefore build it below the tile IR, as an MLIR pass in `libtriton` guarded by a soundness verifier. It gives 2.3–6.7× on control-bound kernels and 1.6–3.8× on an A100 from the same wheel, and ≈1.0× on SpMV and MoE, which carry larger stragglers still but a per-iteration gather that dwarfs the tax. Run on that same distribution with a cheap body the pass gives 6.3×, so the scope condition is a divergent trip count over a cheap body, measured rather than argued. A single-predictor straggler model predicts the cost with $R^2 = 0.997$, and out-of-sample to 1.5%.

CCS Concepts: • **Software and its engineering** → **Compilers**; *Parallel programming languages*; • **Computer systems organization** → *Single instruction, multiple data.*

Keywords: GPU compilers, tile DSLs, Triton, SIMT, warp-level parallelism, irregular parallelism, per-lane control flow

1 Introduction

Tile abstractions have become the default way to write GPU kernels. A Triton program declares tiles and lets the compiler choose the thread mapping [24], and the same bet is now being made in the vendor's own stack [18]. On dense tensor work the bet pays: tile kernels match, and often beat, hand-tuned CUDA.

On irregular kernels they do not. A work-efficient NFA worklist written in Triton runs an order of magnitude below the equivalent CUDA. That much is folklore. Two things make it worth more than a footnote. The kernels it afflicts are not a backwater but where the field is heading, and we measure the same effect on mixture-of-experts routing and ragged attention as on automata. And the constraint is being designed in rather than designed out: NVIDIA's own tile model documents that a tile kernel "follows a single control flow path per block" and forbids tile and `__device__` functions from calling one another [18], so a tile kernel cannot even drop to SIMT for its irregular part.

The question underneath is about language design, and to our knowledge it has not been answered mechanistically: is the gap a *tuning* deficit, which a better autotuner closes, or is it something the abstraction *forecloses*, which no amount of tuning can reach?

This paper answers it. The gap is structural, the missing primitive can be named, and it can be supplied inside the compiler itself.

One shared latch. The mechanism is simpler than the symptom suggests. A tile program's lanes advance in lock step under a single instruction stream, and in particular they share one loop latch. Two consequences follow, and between them they account for the whole residual. A data-dependent loop runs for as long as its warp's *slowest* lane, so the active lanes do full-width work the retired ones no longer need. And a next-state load that each lane depends on cannot be overlapped with its neighbours', so the warp cannot use its lanes as independent generators of in-flight requests [25]. This is not branch or memory divergence [7]: the lanes may be perfectly coalesced and still wait together.

Measured, not asserted. On the NFA worklist the gap decomposes, staged, into a launch-configuration artifact, a redundancy that lane-packing removes, and an irreducible residual (Fig. 1a). Nsight localizes that residual precisely. At matched occupancy, issuing *fewer* warp-instructions than CUDA, and sitting far below both the issue and the bandwidth ceiling, the tile kernel spends 15.3× more cycles in the dependent-load stall. If the mechanism is latency, it should vanish when latency stops being the binding constraint. It does: on a memory-bound DFA the residual closes once the table spills past L2.

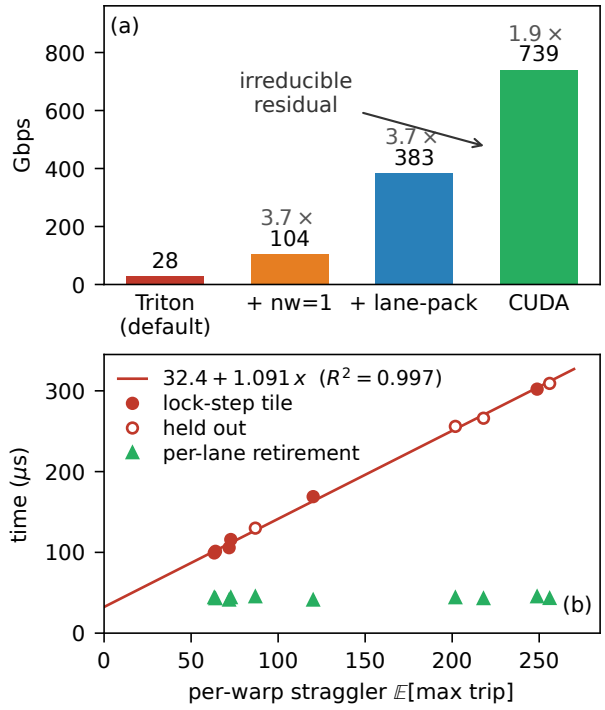

Figure 1. The lock-step tax, and its removal. **(a)** The NFA worklist gap (batch 16384, ≤ 64 states) is not a monolith: a launch-configuration fix and a kernel rewrite recover 3.7× each, and an irreducible residual remains. **(b)** That residual is the per-warp *straggler* tax. Lock-step time grows linearly with $\mathbb{E}[\max \text{ trip}]$; the line is fitted on six designed trip distributions ($R^2 = 0.997$) and the four open markers are held out from it. Per-lane retirement collapses the same times to a flat, distribution-independent floor. All points oracle-gated; every number traces to a versioned CSV.

The primitive, and a wall. The fix the diagnosis asks for is a per-lane loop exit. TritonGPU cannot express it, and not for want of a layout: the carried tensors are already one element per lane. `scf.condition` takes a single `i1`, so a per-lane termination condition is rejected by the verifier. Per-lane retirement is *inexpressible* in structured tile control flow, which is a sharper statement than "Triton is slow here" and a falsifiable one.

The cure, one level down. Since the tile IR cannot host it, we build the primitive underneath, as an MLIR pass in `libtriton` wired into `make_llir`. After lowering, the per-lane predicate already exists; the lock-step latch merely warp-reduces it away. The pass branches on the per-lane predicate instead, guarded by a static verifier that proves observational equivalence before firing. Rebuilt from a pinned one-command recipe, it is oracle-correct and gives 2.3–6.7× on control-bound kernels, 1.6–3.8× on an A100 from the same wheel, and ≈1.0× on gather-bound SpMV and MoE. That last number is not a disappointment but a prediction met: the recoverable cost scales with per-step *control*, not with memory irregularity.

Predictable a priori. Finally the tax is not merely explicable after the fact. It is set by the per-warp straggler, $\mathbb{E}[\max \text{ trip}]$, a quantity a programmer can estimate before writing the kernel. A single-predictor fit predicts lock-step time with $R^2 = 0.997$ and, with coefficients frozen, predicts four held-out distributions to 1.5% (Fig. 1b). The natural competing intuition, that cost tracks the *ratio* of straggler to mean, is wrong, and we falsify it.

Contributions.

1. An instruction-level anatomy of the tile-versus-thread gap on irregular control flow, cross-validated on two architectures, that separates what tuning recovers from an irreducible residual and attributes the residual to abstraction-denied intra-warp latency hiding rather than to divergence, occupancy, instruction count, or bandwidth (§3–§4).
2. A structural impossibility result: per-lane loop termination cannot be expressed in TritonGPU's tile control flow, shown by a verifier-rejected rewrite. We do not stop at naming the missing primitive but specify it, including the body restriction it forces and the one composition question it leaves open (§5).
3. That primitive, built: a soundness-verified MLIR pass in `libtriton` that restores per-lane retirement below the tile IR, oracle-correct, reproduced on four GPUs from one pinned wheel, confirmed at the PTX and SASS level, and driven by a detector in the same compiler (§6).
4. A predictive model of the cost and a generality law with correct negative controls across eight oracle-gated workloads, including a case where the sign inverts and the tile legitimately wins, and one piece of directly actionable guidance: on irregular work a wider tile is monotonically worse, which reverses the dense-tensor intuition (§7–§8).

2 Background and method

NFA simulation, as a work-efficient worklist that iterates the active set with `ffs` over a bitmask, is control-flow and latency bound. DFA simulation, one dependent table gather `trans[cur*256+sym]` per symbol, is memory bound. Between them they give the two faces we need. CUDA and NVIDIA Warp [17] are thread-SIMT, one thread per string; Triton [24] is tile/SPMD, one program over tiles. Warp is the control that keeps the two axes apart, because it is high-level *and* thread-SIMT at once: on this workload it runs at 0.6–0.9× of hand-written CUDA. Nothing that follows is a claim about abstraction *height*.

Correctness gates speed. Every kernel and every configuration is validated bit-for-bit against a CPU oracle

before any throughput is reported, so a fast wrong kernel cannot enter the record. We report medians over warmup+9 repetitions at a GPU-saturating batch [10], since GPU timings are not Gaussian; decomposition ratios are medians over 3 seeds, with dispersion given on the load-bearing anchors. Every mechanism claim is backed by Nsight Compute rather than wall-clock alone: issue-stall composition (the `long_scoreboard` reason isolates dependent-load waits), achieved occupancy, issue activity, warp- and thread-instruction counts, and DRAM and L2 traffic.

Hardware: RTX 4070 (sm_89, 46 SMs), Triton 3.5.1, torch 2.9.1+cu128, CUDA 13.3, driver 580; the in-`libtriton` passes (§6) use a from-source Triton 3.8. Cross-architecture results are on an A100-SXM4-40GB.

3 Anatomy of the gap

The anchor. A register-resident worklist ($\leq$ 64 states, batch 4096, 12 configurations) gives Triton 22 Gbps against CUDA's 227 Gbps, a median of 10.1× (IQR [9.9, 10.5], range 9.4–12.2×, n=12: 3 seeds × 4 state counts). The gap is batch-dependent, and reaches 26× at a saturating batch as the occupancy-gated component below saturates.

A: a launch-configuration artifact. The Triton worklist defaults to `num_warps=4`, and four warps per program redundantly process the *same* string. Sweeping `num_warps`$\in \{1, 2, 4, 8\}$, `nw=1` beats `nw=4` by 2.77× at batch 4096, 3.44× at 16384 and 3.69× at 65536, each doubling roughly halving throughput. The ladder of Table 1 measures the same stage on its own harness and lands at 3.7×; we quote both rather than pick the flattering one, and take the component to be 2.8–3.7× depending on batch. Either way this is pure tuning, it inflates previously reported worklist gaps, and it must be disclosed rather than absorbed into an abstraction claim.

B: warp redundancy, recoverable by packing. The scalar Triton program executes warp-uniformly. Its `thread_inst_per_inst` is 32.00 against CUDA's 30.34: the same number with the opposite meaning, since CUDA's 32 lanes run 32 *different* strings and Triton's run the *same* one. That is ∼90× more warp-instructions. Lane-packing removes it by placing 32 strings in the 32 lanes, which the shared CSR permits because inner-loop bounds stay scalar. Pure packing recovers 3.2×, 9.8× and 19.4× at batch 4096, 16384 and 65536, climbing toward the ideal 32× as more warps fill the device: the benefit is occupancy-gated.

We first read that 90× redundancy as the bottleneck. It is not. Profiling the lane-packed kernel shows the redundancy falls ∼26× while throughput moves only 3.8×, because occupancy was already hiding most of it. The redundancy is real but not load-bearing, and we report it as such.

Table 1. The staged ladder reproduces on a second architecture. Both columns are the *same* four kernels measured the same way (batch 16384, $\leq$ 64 states, medians over seeds and state counts, every row oracle-gated), so the stages are comparable across devices. The total is architecture-stable; the split between the last two stages is not. Fig. 1a shows the sharper per-lane lane-packed variant, which reaches 383 Gbps but was measured on the 4070 only.

	RTX 4070		A100	
Kernel	Gbps	stage	Gbps	stage
Triton worklist (`nw=4`)	28	—	28	—
+ `nw=1`	104	3.7×	101	3.7×
+ lane-packing	314	3.0×	233	2.3×
CUDA worklist (thread-SIMT)	739	2.4×	705	3.0×
total		**26×**		**25×**

C: what is left. A per-lane Triton worklist, in which each lane runs its own `ffs` and its own CSR gather with no cross-lane reduction and no active-set union, beats the union variant by 1.26× and still reaches only 0.51× of CUDA. That residual, ≈2×, is the subject of the rest of the paper. Raising occupancy does not touch it: sweeping `BLOCK`$\in \{32, 64, 128, 256\}$ makes it *worse* (0.49× → 0.31×), because wider lock-step tiles admit more divergence.

Table 1 puts the three components together and repeats the whole ladder on an A100. The total is architecture-stable, 26× against 25×, and the launch-configuration stage is identical on both devices. The remaining two stages trade places, 3.0× then 2.4× on the 4070 against 2.3× then 3.0× on the A100, which is what one expects when the A100's larger L2 and lower clock move where the occupancy-gated benefit saturates. What does not move is that the ladder ends with a stage the tile model cannot climb.

4 The mechanism

Fig. 2 settles what the residual is. In the profiled configuration the per-lane tile kernel issues *fewer* warp-instructions than CUDA (4.66M vs. 5.07M) at the same occupancy (22.5%), and is nonetheless 3.3× slower.[1] Its issue activity is 9.9% against CUDA's 41%, and it spends 15.3× more cycles waiting on a dependent load. The issue-rate ratio (4.1×) tracks the throughput ratio (3.5×), which is what Little's law predicts when latency is fixed and concurrency is the free variable [11, 25].

The instruction roofline [4] closes the obvious objection, that we simply wrote a worse kernel. Both kernels sit far below the issue *and* the bandwidth ceiling (8.3%/27% for Triton, 32%/3.4% for CUDA). A kernel that is neither instruction- nor

[1] The 0.51× of §3 is the median head-to-head across the $\leq$ 64-state sweep; 3.3× is this specific Nsight-profiled configuration. The residual we attribute is the ∼2×.

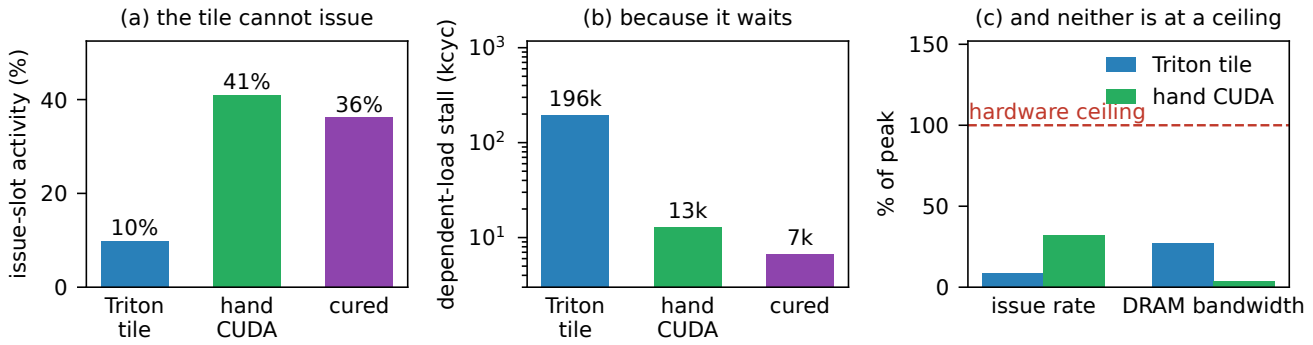

Figure 2. Why the residual is latency and not something cheaper to fix (32 states, batch 16384). **(a)** The per-lane tile kernel occupies its issue slots a quarter as often as CUDA, and per-lane retirement restores it. **(b)** The missing time is the dependent-load stall (`long_scoreboard`), 15.3× CUDA's on the tile and 29× lower again once cured. **(c)** Neither kernel is near a hardware ceiling, so this is not a worse kernel: it is a kernel that cannot keep loads in flight.

bandwidth-bound, issues fewer instructions than its rival, and still runs 3.5× slower, is latency-bound by elimination.

One refinement runs against our own initial reading. We predicted, following the latency-hiding literature, that the tile kernel would sustain *fewer* in-flight requests. It sustains 26–84× *more*, because inactive lanes still issue masked gathers. So the tile pays twice, in excess masked traffic and in an inability to overlap the dependent load, and the mechanism must be stated through stall composition and issue activity rather than request counts.

We also distinguish this from the hardware fix for intra-warp stalls [3]. The same SM is fast under CUDA and slow under Triton at equal occupancy and fewer instructions, so the loss is imposed by the abstraction, not by the machine.

The regime test. A latency explanation makes a falsifiable prediction: where latency is not the binding constraint, the residual should vanish. The DFA supplies the test. Across table sizes the scalar Triton DFA is flat at ~29 Gbps, an independent confirmation of the scalar-gather ceiling, and lane-packing gives ~12× over it. The lane-packed-to-CUDA ratio is 0.55–0.62× while the table is cache-resident but 1.05× at a 16 MB table (Fig. 3): lane-packed Triton *matches* CUDA. DRAM utilization is only 14.6% there, so this is memory *latency*, not saturated bandwidth. At cache latency CUDA's intra-warp hiding wins ~1.7×; at DRAM latency both must hide latency across warps, and they converge. The NFA's irreducible residual and the DFA's closeable gap are one mechanism at two latency scales.

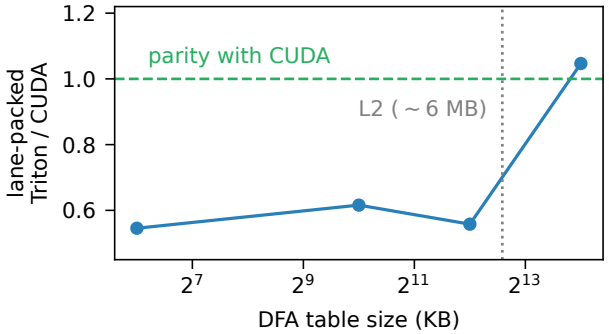

Figure 3. The residual is regime-dependent. Lane-packed Triton trails CUDA by ~1.7× while the DFA table is cache-resident and reaches parity (1.05×) once a 16 MB table spills past L2, where both must hide latency across warps.

5 The wall: per-lane exit is inexpressible

The diagnosis names what a fix must provide: independent per-lane `ffs` and `while`, per-lane data-dependent loop bounds, a register-resident per-lane bitset, and a scalar gather. Call it a per-lane sub-tile loop with an independent exit. The natural place to add it is the tile IR. It cannot go there, and why it cannot is a statement about the abstraction rather than about one compiler's maturity.

We first checked whether a lower-level tile frontend already supplies it. It does not. Triton's Gluon exposes per-thread *layout* through Linear Layouts [32], not per-lane *control flow*: a runnable probe shows `gl.load` returns a layout-typed block and never a scalar, so the data-dependent CSR loop cannot be lowered at all, and we capture the compile error verbatim. Layout control is not control-flow divergence.

Two facts from the matched TritonGPU IR then close the question.

The lock-step is not a layout choice. The tensors carried by the loop are already `sizePerThread=1`, one element per lane. Re-encoding them is a no-op, so no amount of per-thread layout, in Gluon or anywhere else, changes the behaviour.

The lock-step is the loop construct itself. Fig. 4 shows where. The per-lane predicate is computed and then immediately destroyed: a `tt.reduce` collapses the 32-lane comparison to one `i32`, and `scf.condition` consumes a single `i1`. The natural rewrite is to carry the per-lane condition instead, and

```
#blocked = <{sizePerThread = [1],   // already ONE
             threadsPerWarp = [32]}>  // element/lane

%j:2 = scf.while (%acc = %cst, %j = %cst) : (
    tensor<32xi32, #blocked>, ...) -> (...) {
  %2 = arith.cmpi slt, %j, %trip
     : tensor<32xi32, #blocked>   // per-lane predicate
  %5 = "tt.reduce"(%2) <{axis = 0}>  // ...destroyed here
     : (tensor<32xi32, #blocked>) -> i32
  %6 = arith.cmpi sgt, %5, %c0 : i32
  scf.condition(%6) %acc, %j : ...  // ONE i1, 32 lanes
} do {                      // masked full-width body
  %active = arith.cmpi slt, %j, %trip
  %acc_9  = arith.select %active, %j, %cst
  ...
}
```

Figure 4. The lock-step latch, in the matched TritonGPU IR (trimmed; source locations removed). The lanes already hold one element each, so no layout change can help. The per-lane predicate `%2` exists and is then reduced to a scalar, because `scf.condition` admits exactly one `i1` for the whole tile. That single `i1` is the lock-step.

the MLIR verifier rejects it on the type (“*'i1' vs 'tensor⟨8×i1⟩'*”, captured with the built `triton-opt`; the probe uses an eight-lane tile, since the type mismatch is the point and not the width). Per-lane loop termination is therefore *inexpressible* in TritonGPU’s structured tile control flow.

The constraint is not Triton’s alone, which matters because otherwise this would be a statement about one compiler’s maturity. NVIDIA’s tile model documents the same property in its own words, that a tile kernel “follows a single control flow path per block” [18], and forbids tile and `__device__` functions from calling one another, so there is no dropping to SIMT for the irregular part of a kernel.

That is falsifiable and it is actionable. It is falsifiable because adding a per-lane sub-tile loop and exit op to the tile IR would make the statement false, which is precisely the language change we are arguing for. It is actionable because it tells us where the cure has to live: below the tile IR, in the lowering to the thread model.

What the primitive would have to be. Naming a missing op is cheap unless one says what it must mean, so here is the specification our lowering already implies. Take `scf.while`’s condition from `i1` to a tile of `i1` under the same layout as the carried tensors, and read it lane-wise: a lane whose predicate is false takes no further trips and stops updating its slice of the iter-args, and the region exits when no lane is still live. Live-outs freeze, per lane, at the trip on which that lane left. That freezing is what makes the construct observationally equivalent to today’s masked loop, rather than merely faster than it.

The price is a restriction on the body, and it is not incidental. A retired lane cannot participate in a cross-lane operation, so the region may not contain reductions, broadcasts, or tile-wide shared-memory operations. That is exactly the body-safety condition our verifier already discharges (§6), so the analysis a compiler would need in order to admit the op is one we have had to write anyway. Reconvergence for whatever tile operation follows is a barrier at the join, which is the `bar.warp.sync` our pass inserts.

What stays genuinely open is composition with tile operations that assume full width, `tt.dot` above all. The op is an escape hatch for scalar control and has no business inside a matmul, so the question is where a language draws that boundary, not whether the lowering works.

6 Per-lane retirement, built

6.1 Detecting the region

On a from-source Triton (3.8.0) we add a TritonGPU pass, `tritongpu-thread-region`, that runs in the NVIDIA `make_ttgir` pipeline and recognises the lock-step signature: an `scf.while` whose iter-args are `#blocked` tile tensors and whose `scf.condition` derives from a `tt.reduce` to scalar, which is exactly “the whole tile loops until the busiest lane is done, and idle lanes are masked”. It compiles into `libtriton` and fires on the target kernels; with the pass off the attribute is absent and codegen stays bit-exact.

6.2 What can be fixed inside the tile IR, and what cannot

Part of the cost is reachable without leaving the tile IR. The per-iteration `tt.reduce` that gates the lock-step `while` is loop-invariant in disguise: the lane counter is a uniform splat, so $\mathrm{any}(j < \mathit{trip})$ equals $j < \max(\mathit{trip})$. We extend the pass, behind a flag, to hoist `reduce_max(trip)` once before the loop and replace the per-iteration cross-lane reduce with a scalar counter, leaving the masked body untouched. It is provably equivalent, preserves per-warp termination, is verified bit-exact end-to-end, and is 1.55× faster on the lock-step kernel ($155.6 \rightarrow 100.4\,\mu s$; detection-only mode is a performance no-op, which confirms the gain is the rewrite and not the pass machinery).

This is an honest partial. It removes the reduce overhead. It does not grant per-lane retirement, because §5 says it cannot.

6.3 The full primitive, below the tile IR

After `convert-triton-gpu-to-llvm` the per-lane predicate already exists as a scalar $j_{\mathrm{lane}} < \mathit{trip}_{\mathrm{lane}}$, and the lock-step latch merely warp-reduces it (`nvvm.redux.sync`) into a uniform `i1` that gates `llvm.cond_br`. That is the whole lock-step, in one instruction, at a level where we can reach it.

Our pass, a real MLIR pass in `libtriton` wired into `make_llir`, redirects the branch to the per-lane predicate so each lane retires independently under hardware Independent Thread Scheduling [19], deletes the now-dead cross-lane

reduce, and inserts a `bar.warp.sync` at the reconvergence block.

The rewrite is visible all the way down. In PTX the masked baseline reduces the active mask (`redux.sync.max.s32`) and branches the warp on the resulting uniform predicate, whereas the cured code branches on this lane's own `setp.lt.s32` and carries a `bar.warp.sync` at reconvergence (`redux.sync` $1 \to 0$, `bar.warp.sync` $0 \to 1$). It survives `ptxas` to SASS: the warp-collective `REDUX.MAX.S32` into a uniform register disappears (`REDUX` $1 \to 0$, static instructions $48 \to 40$) and the latch branches on the per-lane `ISETP`, with reconvergence realized by the standard `BSSY`/`BSYNC` barriers. Per-lane retirement is in the machine code, not only in the IR.

6.4 Sound by default

A rewrite that changes when lanes stop executing is only safe under conditions we can check, so the pass fires only behind a static verifier that proves observational equivalence: body safety (no cross-lane op anywhere in the loop body), a single exit, live-out freezing through the lowering's struct projections, and predicate *monotonicity*, which live-out freezing alone does not imply. Where the proof fails, the pass declines.

That is not hypothetical. On a rejection-sampling kernel whose latch is an accumulate-OR early exit rather than a counter-versus-bound compare, the verifier cannot establish monotonicity and declines to fire, on both GPUs, leaving the kernel bit-exact (Table 2). We would rather report a decline than a rewrite we cannot justify.

6.5 What it recovers

Compiled and run end-to-end from a pinned recipe (a base commit plus a versioned patch, built to a wheel), the pass is oracle-correct and gives 2.3–6.7× across six trip distributions on the RTX 4070 (geometric law: median 2.46×, IQR $[2.46, 2.50]$, n=5 seeds) and 1.6–3.8× on an A100 from the *same* wheel. On both GPUs the cured time collapses to a flat, distribution-independent floor ($43.0 \pm 1.6\ \mu$s and $\sim 75\ \mu$s), which is the mechanism's signature: once each lane retires at its own trip count, the distribution stops mattering.

These six are a synthetic kernel with injected trip divergence. They isolate the mechanism and *upper-bound* the payoff; they are not a general speedup claim. Nsight attributes the gain to work, not occupancy: issued instructions fall 39× (36.1M → 0.92M) because total work becomes $\sum_i trip_i$ rather than 32× the busiest lane, while threads-per-instruction is unchanged.

Which cost this closes, and which it does not. The residual of §4 and the tax of §7 both follow from the shared latch, but they are not the same cost and the pass addresses one of them. Per-lane retirement removes the work the lock-step forces on lanes that should already have stopped, which is why Nsight reports its gain as instructions rather than stall cycles. Restoring intra-warp latency hiding additionally requires the lanes to be running independent work, which is what the thread lowering below does and what the tile IR withholds for the reason of §5. We measured each with the metric that applies to it, and do not claim the pass was shown to do the other's job.

The pass also fires, oracle-correct, on real workloads carrying the same latch: power-law CSR SpMV and MoE top-k routing, with `redux.sync` removed and `bar.warp.sync` present in the emitted PTX. Both carry a per-iteration memory gather, so almost none of their cost is the control reduce, and the gain is correspondingly ≈1.0× on both GPUs.

The obvious reading of that is wrong, and the difference matters. These workloads do not lack stragglers: the power-law CSR matrix the SpMV witness runs on has a per-warp straggler of 150 rows against a mean of 16, *larger* than five of the six synthetic distributions. The lock-step tax is present and large in iterations. What differs is the denominator, because a DRAM gather costs so much more per iteration than the control beside it that removing the tax moves the total by noise.

The condition, separated experimentally. Distribution and body are independent variables, so that is testable rather than arguable. We ran the lock-step kernel on the *same* power-law row lengths, with an accumulate body in place of the gather. Same input, same straggler, different denominator: the pass gives **6.31**× where SpMV gets ≈1.0×, while uniform rows, the matched control with no divergence, give 1.09×. So the condition is not a divergent trip count, which SpMV has in abundance, but a divergent trip count *over a cheap body*. This is a controlled factorization and not a real-workload speedup, the body being synthetic on purpose because it is the variable held. These runs are on an *H100*, a third architecture, where the geometric law gives 1.47× against 2.46× and 1.8× with the PTX evidence unchanged.

The condition is also met outside our own microbenchmarks. Stein's binary GCD, the subtract-and-shift loop crypto libraries use, and the Collatz step count both have data-dependent trip counts over bodies with no memory traffic at all, and on both the pass fires and pays (1.27× at straggler 47, 1.79× at straggler 215, ordered by absolute straggler as the law requires). A cheap body is not a free one, so these are modest beside the synthetic range; what matters is that the gather-bound workloads, whose stragglers are *larger*, get nothing.

6.6 An out-of-band bound, and the loop closed

Two results sit beside the in-compiler pass, and which is which matters.

The bound. Out of band, we lower the *same* idiomatic per-lane source, the active-set `ffs` worklist with its data-dependent CSR loop, to a per-thread CUDA kernel through `nvcc`. Oracle-validated on no-accept automata, which

Table 2. The built pass, rebuilt from one pinned wheel and run on four GPUs. Speedup is against the masked tile baseline; every row is oracle-correct, and every row traces to a versioned CSV. The benefit scales with control-boundedness, and the verifier declines the out-of-scope latch rather than rewriting it. †measured on an H100 from the same wheel, where the geometric law gives 1.47×. All three are divergent trips over a cheap body: the first is the SpMV row-length distribution itself, run over an accumulate instead of the gather, and is the controlled counterpart to the SpMV row below.

Workload	Bound by	RTX 4070	A100
Lock-step `while` (6 dists)	control	**2.3–6.7×**	1.6–3.8×
geometric (n=5 seeds)	control	2.46×	1.8×
single-straggler (held out)	control	7.2×	n/a
real SpMV rows†	control	6.31×	n/a
Binary GCD, Stein†	control	1.27×	n/a
Collatz step count†	control	1.79×	n/a
MoE top-k routing	gather	≈1.0×	≈1.0×
SpMV CSR (power-law)	gather	≈1.0×	≈1.0×
Rejection sampling	control	*declines*	*declines*

removes the early-exit confound, that lowering runs 4.2× the tile lowering (≈1555 vs. ≈378 Gbps) and exceeds hand-written CUDA (2.15×, the generated kernel being a minimal thread worklist). It is an existence proof and an upper bound, not the contribution: the front-end source was never the problem, the tile lowering is. The second number cuts our way and we would rather state it than let it be found: our hand-CUDA baseline is evidently not the thread model's ceiling, so every gap this paper reports against it is a *lower* bound on what the tile lowering forgoes. It also acts through the diagnosed mechanism, restoring issue activity to 36% against the tile's 9.9% and CUDA's 41% and cutting the dependent-load stall 29× (Fig. 2).

The loop. A selector then closes the pipeline. It runs the detection pass of §6, and where the lock-step signature is present routes the region to *that out-of-band lowering*, otherwise leaving it on the tile path. Oracle-gated, the NFA worklist is auto-routed for 3.9× over the tile across a 16–64 state sweep, consistent with the 4.2× above, while a fixed-trip negative control is detected-negative and stays on the tile path. Detect, decide, lower, measured, end to end.

The loop, without the seam. That selector routes to an out-of-band lowering, so the decision and the rewrite live in different toolchains. They need not. Built into one wheel, the detection pass marks the region in `make_ttgir` and the retirement pass rewrites its latch in `make_llir`, so the whole decision runs in one process against one compiler: read the attribute, and recompile with retirement where it fired. On an H200 the lock-step worklist is detected and routed for **1.90×** (69.8 → 36.7 μs, `redux.sync` 1 → 0), while a fixed-trip

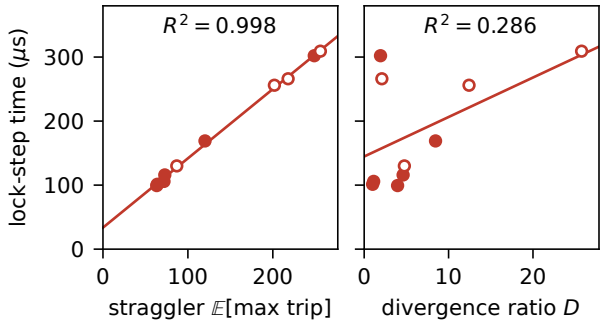

Figure 5. The lock-step cost is set by the absolute straggler, not by the divergence *ratio*. Ten trip distributions, filled markers designed, open markers held out. A linear fit on the straggler alone explains the cost; the same fit on $D = \mathbb{E}[\max\ \text{trip}]/\mathbb{E}[\textit{trip}]$ does not.

variant of the *same* body is not detected and keeps the tile path. Asking the detection pass for its in-IR rewrite mode instead changes neither decision nor timing. The kernel here is the lock-step worklist rather than the full NFA, so this closes the loop and not the workload.

7 The tax is the straggler

A diagnosis is worth more if it predicts. A controlled sweep over six trip distributions (constant, low-variance, wide-uniform, geometric, and two Pareto laws; 2^{20} elements each, oracle-gated in both modes) pins the masked baseline to a single *a-priori* quantity, the per-warp straggler $\mathbb{E}[\max_{\text{warp}} \textit{trip}]$. Least squares gives

$$t_{\text{masked}} = 32.4 + 1.09\ \mathbb{E}[\text{warp-max}]\ \mu\text{s}, \qquad R^2 = 0.997,$$

which is what a latch that iterates to the busiest lane must do. The cured kernel collapses to the flat floor of §6 because each lane retires at its own trip count. The slope is the kernel's per-iteration lock-step cost and the intercept is compiler overhead.

The intuitive predictor is the wrong one. It is natural to expect the cost to track the intra-warp divergence *ratio* $D = \mathbb{E}[\text{warp-max}]/\mathbb{E}[\textit{trip}]$. It does not. Fitting the same masked times against D gives $R^2 = 0.00$ over the six designed distributions and only 0.29 once the four held-out ones are added, against 0.997 and 0.998 for the straggler on the same two samples (Fig. 5). The concrete inversion makes the point: the geometric law ($D = 3.96$) yields 2.3× while wide-uniform ($D = 1.94$) yields 6.7×, a *lower* ratio but a much larger absolute straggler, hence more lock-step waste. What is wasted is warp-cycles, and warp-cycles are counted in absolute iterations.

Out of sample. With the coefficients frozen, the model predicts four *held-out* distributions generated from a different seed (bimodal, log-normal, spike, and an adversarial

single-straggler warp in which 1/32 lanes run 256 iterations and the rest run 2) to within 1.5% on average and 2.1% at worst on lock-step time, and every in-sample speedup to under 8%. That last case yields 7.2×, the effect at its purest: one slow lane taxing the other 31, which is what per-lane retirement removes. It is also the least artificial of the six, being the shape of any batch that holds one long sequence.

What the law is, and is not. The validation is across trip *distributions* of one kernel, not across kernels, and the two coefficients are that kernel's own: change the loop body and both move. What transfers is the *form*, that cost follows the absolute straggler rather than the divergence ratio, and two things support it beyond one good fit. The coefficients are not free parameters but have a mechanical reading, and the slope survived a rebuild of the pass that moved the intercept (1.08 against 1.09, intercept 50 → 32 μs). So a programmer gets the right *shape* of estimate before writing a kernel and calibrates the constant on their own body, which is still better than the qualitative "divergent trips cost more" it replaces.

8 Generality, and where it stops

Is any of this specific to automata? To find out we built six further oracle-gated tile-versus-thread witnesses spanning the irregularity space, each with identical machinery (a Triton tile kernel, the same per-lane source lowered to CUDA threads, and a CPU oracle), and then two more drawn from ML. Fig. 6 reports the result. The law is that **the regret that grows, and that the cure recovers, is created by per-step scalar control rather than by memory irregularity**. It is deliberately not the stronger claim that all regret is, because a tile mapping can also lose occupancy and charge a floor with no divergence anywhere, which we measure below.

The negative control. A graph pointer-chase, one million random walkers with data-dependent gather addresses over scattered `colidx` at degree CV 3.79, but a *fixed* step count, is the decisive case. Tile and thread are indistinguishable in issue activity (5.42% vs. 5.43%), occupancy (72.1% vs. 72.1%) and threads-per-instruction (32 vs. 32), and the regret is 1.00×. Irregular memory and dependent loads on their own cost nothing, because a lock-step gather already supplies full intra-warp memory-level parallelism.

Two channels, not one. We first stated the law as a single "tile issue deficit" predictor. Completing the Nsight profiling falsified that, and the honest form has two channels. *Issue starvation*: a data-dependent scalar recurrence drives the tile's issue rate below the thread's, as in the NFA worklist (1.96× at 9.9% vs. 41%). *Masked-lane waste*: divergent trip counts make the tile do full-width work where the thread retires lanes, as in rejection sampling (4.0×, tile threads-per-instruction 32 against the thread's 17) whose issue rate is if anything *above* the thread's, so the waste is in what the active lanes compute. The floor is the third component, and it is divergence-free: uniform-`nnz` SpMV has zero divergence and still costs 1.94×, because the tile mapping loses occupancy against the thread mapping on a bandwidth-bound gather. It is not universal, since the pointer-chase pays no floor at matched occupancy, but it is real where the mapping costs occupancy. That falsified our initial guess that it would be ≈1×, and it is why the law above is scoped to the regret that *grows*. At a fixed tile width the divergence increment sits on top of it: on the A100 at width 32, the zero-divergence matrix costs 1.80× and the power-law one 3.20×. So the law is multi-component, and only its variable part is what per-lane retirement can return.

We had also held that widening the tile amplifies that increment. It does not. Across widths {32, 64, 128, 256} on the A100 the *zero-divergence* matrix climbs 1.80 → 5.64 → 5.65 → 5.73× while the divergent one climbs only 3.20 → 3.94 → 4.57 → 5.60×, so by width 256 the matrix with *no* control divergence is the dearer of the two. What grows with tile width is the lowering baseline, not the divergence cost, and part of it is the launch-configuration artifact of §3, since the width sets `num_warps`. The components are separable but not additive in width.

Read together with the `BLOCK` sweep of §3, where widening the NFA tile takes it from 0.49× to 0.31× of CUDA, this is the one piece of directly actionable advice the paper carries: on irregular work a wider tile is monotonically worse, on two workloads measured one on each of two architectures. That is the reverse of the dense-tensor intuition, where a wider tile amortizes, and it follows from the mechanism. A wider tile puts more lanes under one latch, so it buys more of exactly the thing that costs.

It reaches ML, including where it reverses. Two witnesses on the kernels tile DSLs are actually built for. *MoE top-k routing*, with ragged data-dependent expert counts and a scalar per-token accumulate, is the ML face of scalar control: regret 2.36×, the tile issuing 2.6× more instructions (41.8M vs. 16.3M) and doing masked full-width work (threads-per-instruction 30.1 vs. 7.7). *Ragged variable-context attention* has the *identical* ragged structure but a *dense* head-dim payload per step, and here the law predicts a *negative* and is right: regret 0.64×, the tile wins. The thread model retires lanes in both cases (threads-per-instruction 3.45 on attention against 7.7 on MoE); what flips the sign is per-step instruction efficiency. Scalar work makes the tile issue more, through the lock-step reduce and masking, so it loses; a dense vectorizable head-dim makes it issue fewer (123M vs. 178M) so it wins despite no lane retirement.

That is why Triton is the tool of choice for flash-attention and collapses on automata, and it is a scope statement as much as a generalization. These witnesses use the framework's one-element-per-lane mapping; production attention is warp-cooperative, a different design point, and whether the law survives there is future work.

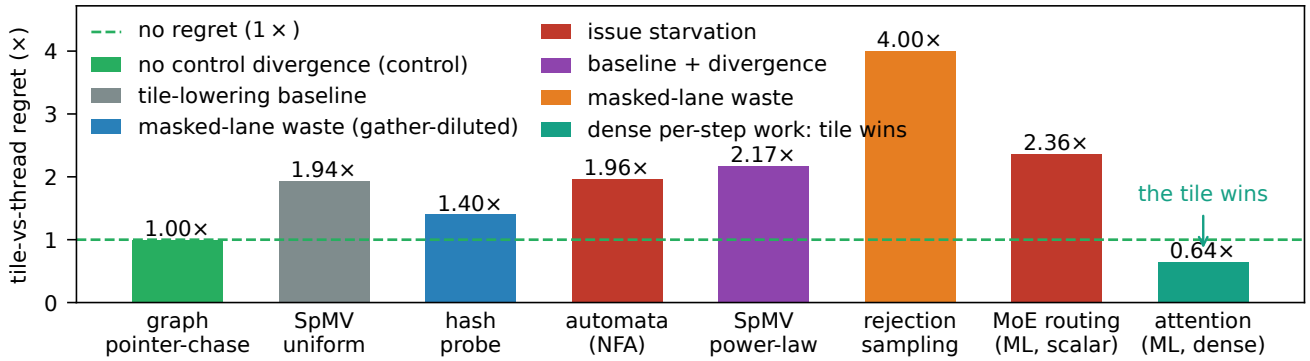

Figure 6. Tile-versus-thread regret across eight oracle-gated irregular witnesses, coloured by dominant mechanism. The graph pointer-chase is the negative control: irregular memory and dependent loads with a *fixed* step count cost nothing at all. Uniform SpMV is the other half of the story: no divergence either, but the tile mapping loses occupancy, so a floor can exist without any control cost. What grows *above* such a floor is set by per-step scalar control. Ragged attention inverts the sign, because a dense head-dim payload makes the tile issue *fewer* instructions than the thread model.

9 Threats to validity

One GPU, mostly. The primary results are on an RTX 4070. The *mechanism* is a property of the SIMT execution model and the tile-versus-thread lowering rather than of a device, so we test it directly: a one-command harness reruns every witness on a new GPU, tags output by device, and checks the falsifiable prediction that each regret persists in *direction* while magnitudes rescale. On an A100-SXM4-40GB (sm_80, 6.7× the L2) all five portable witnesses confirm it with genuinely re-measured, architecture-different magnitudes, from hash-probe (1.40 → 1.48) to power-law SpMV (2.17 → 3.20), while the memory-only control stays paradigm-neutral (0.99 → 0.99). The flagship ladder reproduces too (Table 1), as does the built pass (Table 2). We report the ladder's stage factors from the two devices' own measurements rather than asserting they match: the total holds to within 3% while the internal split shifts, and that is the honest form of the claim. The batch-4096 anchor rescales 10.1× → 5.6×, because that register-resident regime is clock-sensitive and the A100's lower clock compresses both sides; the direction persists.

Where the pass applies. The lowering is a real MLIR pass in `libtriton`, but it fires only on the `tl.max` lock-step latch, and the body-safety guard bails when the loop carries a cross-lane op. That the cure must live below the tile IR is a property of *today's* tile IR, and §5 states exactly the language change that would move it up.

Regime of the clean residual, and where the ordering reverses. The clean ~2× residual is a register-resident (≤ 64-state) result, and past that boundary the head-to-head does not merely blur, it *inverts*. The CUDA register worklist is flat at ~227 Gbps across the whole single-word range (16 to 64 states, batch 4096) and then falls to 26.9 Gbps at 96 states, an 8.5× cliff at exactly the point where its working set needs a second 64-bit word; it declines again to 15.2 and 13.3 Gbps at four words. The degradation tracks *word count*, not state count, which is the signature of register pressure rather than of growing work: 96 and 128 states differ by a third in size and by under two percent in CUDA throughput. The multi-word lane-packed tile kernel crosses the same boundary far more cheaply, so it is 1.44–2.15× *faster* than hand-written CUDA from 96 to 256 states, including at 192 states where our power-of-two tile width makes it carry four words against CUDA's three.

This bounds the claim rather than decorating it: regret follows the execution paradigm within the regime where that paradigm's resources hold, and the thread-SIMT baseline surrenders its advantage exactly when a fixed register file stops holding the working set. Two caveats belong with the numbers. These four points are one seed each, against three or more elsewhere, and they are batch 4096, where the residual of §3 is 1.47× rather than ~2×. Above 512 states the register baseline cannot be built at all, the bit-packed representation capping at eight words, so ANMLZoo-scale automata such as Brill (42661 states, 667 words) have no thread-model baseline to be compared against and run oracle-valid but sub-Gbps, an *algorithmic* cost orthogonal to the regret.

Tuning, not expressibility. `num_warps` is a knob, and the strongest counter-thesis is that this is all under-tuning [21]. We disclose the full sweep rather than one configuration, which separates the two: tuning closes component A and part of B, while the residual C is flat across the sweep. The Triton/Gluon pair adds a control with the same MLIR stack in which only the expressivity lever moves, and §5 converts the claim into a verifier-checkable statement rather than a judgement call.

On our own corrections. Five hypotheses of ours were wrong, and we report each where it arose rather than collecting them in a footnote: warp redundancy as the bottleneck (§3); a request-count deficit as the mechanism (§4); a single-channel issue-deficit law, uniform SpMV at ≈1× when it is 1.9×, and tile width amplifying the divergence increment when it amplifies the baseline instead (all §8). Every one of them narrowed a claim, and we prefer that record to a cleaner-looking one.

10 Reproducibility

Every number here traces to a versioned CSV, and all figures regenerate from those CSVs through a single script, so plots cannot drift from measurements. An artifact index maps each claim to the command that produces it and the CSV it writes.

The expressibility and structural claims are *runnable, falsifiable probes* rather than prose: the Gluon probe exits 0 on the expected compile failure and 1 if some future Gluon compiles it, and the lowering-wall probe asserts that the MLIR verifier rejects a per-lane `scf.condition`. The per-lane-retirement pass builds as a wheel from a one-command recipe, a pinned upstream commit plus a versioned patch, and every cure number here came from that wheel, on all four GPUs.

11 Related work

GPU automata form an *algorithmic* axis, all single-implementation CUDA. The nearest neighbour is Liu et al. [14], who ask why a GPU is slow at NFAs and answer with data movement and compute utilization, then privatize reads into on-chip memory. Ours is the adjacent question with an answer of a different kind: why the *tile DSL* is slow at NFAs when hand-written CUDA on the same GPU is not. The two are complementary, and our pointer-chase control sharpens the boundary, since irregular memory alone carries no tile-versus-thread cost at all (§8). The rest of the axis has the same shape: ngAP's non-blocking multi-symbol processing [9], HybridSA's bit-parallel shift-and [13], BitGen's Parabix bitstreams [8], AsyncAP's input-symbol asynchrony [15], AutomataBLAS's AP-as-SpMV [30], and the Hopps sparsity ASIC [5]. Each varies the algorithm on one implementation; none compares programming models or holds the algorithm fixed across them. The sole IR-for-automata work [23] lowers regex to one fixed domain-specific accelerator, a hardware target rather than a GPU-paradigm study, with no tile-versus-thread cost. Our axis is orthogonal: fixed algorithm, varied programming model.

Tile GPU DSLs. Triton [24] and the 2024–26 wave exposing more thread and warp control. The closest, Gluon's Linear Layouts [32], gives per-thread *data placement* but not per-lane *control flow*, as our probe shows; Descend [12] makes the thread hierarchy first-class but offers no tile drop-to-scalar escape; Warp [17] is the thread-SIMT existence proof. TileLang [27] and Hexcute [31] push further into the same space, the first decoupling schedule from dataflow and the second synthesising layout and task mapping by type inference, but both allocate *work and data* to threads rather than giving a lane control flow of its own. The tile direction is being invested in beyond CUDA C++ as well, including an ownership-typed Rust frontend over the same tile model [6], a safety axis orthogonal to what a tile can express. ML-Triton [26] is the clearest sign of the trend and of its stopping point: it moves Triton's interface one level finer, adding warp-level programming and a workgroup→warp→intrinsic lowering, and reaches 95% of expert-written kernels on dense GEMM and attention. A warp is still 32 lanes under one latch, so it halts exactly one level above where the cost we measure lives.

Mixing thread- and tile-level execution is the live frontier, and every approach is programmer-directed or differently directed. Prism [1] makes thread granularity part of the type system so group-coordinated code composes safely, but the granularity is still the one the programmer writes, and nothing diagnoses that an existing tile kernel is paying a lock-step tax. NVIDIA's cuTile [18] is the strongest evidence that the constraint is designed in rather than incidental. Its programming guide states that a tile kernel "follows a single control flow path per block", restricts which control-flow constructs a tile kernel may use at all, and forbids `__tile__` and `__device__` functions from calling one another, so a tile kernel cannot drop to SIMT for its irregular part. The gap we name therefore persists in *both* TritonGPU and Tile IR, and the primitive we propose is complementary to that direction rather than subsumed by it. Insum [29] generates sparse GPU kernels from indirect Einsums and reaches 1.14–3.81× over hand-written implementations, but on the format-and-representation axis, lowering through the PyTorch compiler; it does not touch per-lane control flow, and a tile lowering underneath it would pay the same tax. Tawa [2] auto-restructures Triton but for *dense* warp specialization. Partial control-flow linearization [16] runs the *opposite* way, vectorizing divergent CPU control flow, where we de-vectorize a tile region to recover per-lane memory-level parallelism. None automatically detects an irregular data-dependent per-lane region in a tile DSL and lowers it to the thread model, and our structural result says why an in-tile-IR rewrite cannot suffice.

Mechanism. We ground the residual in latency-hiding and MLP-versus-occupancy analysis [11, 25] and place both kernels on the instruction roofline [4]; we distinguish it from divergence [7] and from the *hardware* fix of Subwarp Interleaving [3], attributing the same stall to the DSL at equal occupancy and fewer instructions. Compiler work on divergence reduces it by melding similar control flow [22] or by linearizing it [16]; both keep one instruction stream per warp, where we hand the lanes their own.

Methodology and lineage. Rigorous benchmarking [10], roofline [28], and the performance-portability metric [20], which is per-hardware where we measure per-*capability*; we answer the autotuning counter-thesis [21] with the same-MLIR control and the disclosed sweep. The gap we fill is a constructive, instruction-level account of *why* a tile DSL pays on irregular control flow, which names the missing primitive and builds it.

12 Conclusion

The cost a tile DSL pays on irregular control flow is not a monolith and not a mystery. Two stages of tuning recover most of the headline gap, and what remains is one shared loop latch: lanes that cannot retire independently, and dependent loads that cannot be overlapped across them. That residual vanishes for a DRAM-resident DFA, where latency stops binding, and its magnitude is set by the per-warp straggler, so it is predictable before a line of kernel code is written.

The primitive this asks for is a per-lane sub-tile loop with an independent exit. Today's tile IR provably cannot express it, so we built it underneath: a soundness-verified MLIR pass in `libtriton`, oracle-correct on four architectures and visible in the emitted SASS. The remaining step is a language one. Add the loop and its exit to the tile IR itself, and the wall this paper documents ceases to exist.

Acknowledgments

Disclosure of generative AI use, per the ACM Publications Policy on Authorship. We used a generative AI coding assistant (Anthropic's Claude, driven through an agentic command-line interface) extensively in producing this work. It wrote and refactored the bulk of the kernel, harness, and analysis code, including the compiler passes described in Section 6; it ran the measurement scripts; it generated the figures from the versioned CSVs; and it drafted and revised the text of this paper. The research questions, the hypotheses, the experimental designs, and every scientific claim are ours, as are the decisions about what to keep, retract, or re-measure. No number in this paper is assistant-generated prose: each one is a recorded measurement in a versioned CSV, and each kernel is correctness-gated against a CPU oracle, so the results stand or fall on the artifact rather than on the assistant. We have verified the content and take full responsibility for it.

References

[1] Manya Bansal, Daniel Sainati, Joseph W. Cutler, Saman Amarasinghe, and Jonathan Ragan-Kelley. 2026. Modular GPU Programming with Typed Perspectives. *Proc. ACM Program. Lang. (PLDI)* 10, PLDI (2026), 1077–1101. arXiv:2511.11939. doi:10.1145/3808290

[2] Hongzheng Chen, Bin Fan, Alexander Collins, Bastian Hagedorn, Evghenii Gaburov, Masahiro Masuda, Matthew Brookhart, Chris Sullivan, Jason Knight, Zhiru Zhang, and Vinod Grover. 2026. Tawa: Automatic Warp Specialization for Modern GPUs with Asynchronous References. CGO; arXiv:2510.14719. https://arxiv.org/abs/2510.14719

[3] Sana Damani, Mark Stephenson, Ram Rangan, Daniel R. Johnson, Rishkul Kulkarni, and Stephen W. Keckler. 2022. GPU Subwarp Interleaving. In *HPCA*. IEEE, Piscataway, NJ, USA, 1184–1197. doi:10.1109/HPCA53966.2022.00090

[4] Nan Ding, Muaaz Awan, and Samuel Williams. 2022. Instruction Roofline: An Insightful Visual Performance Model for GPUs. *Concurrency and Computation: Practice and Experience* 34, 20 (2022), e6591. doi:10.1002/cpe.6591

[5] Xingran Du, Joel S. Emer, and Daniel Sánchez. 2025. Hopps: Leveraging Sparsity to Accelerate Automata Processing. In *ASPLOS*. ACM, New York, NY, USA, 96–111. doi:10.1145/3676642.3736126

[6] Melih Elibol, Jared Roesch, Isaac Gelado, Eric Buehler, and Michael Garland. 2026. Fearless Concurrency on the GPU. arXiv:2606.15991. https://arxiv.org/abs/2606.15991

[7] Wilson W. L. Fung, Ivan Sham, George Yuan, and Tor M. Aamodt. 2007. Dynamic Warp Formation and Scheduling for Efficient GPU Control Flow. In *MICRO*. IEEE Computer Society, Washington, DC, USA, 407–420. doi:10.1109/MICRO.2007.30

[8] Tianao Ge, Xiaowen Chu, and Hongyuan Liu. 2025. Interleaved Bitstream Execution for Multi-Pattern Regex Matching on GPUs. In *MICRO*. ACM, New York, NY, USA, 385–400. doi:10.1145/3725843.3756052

[9] Tianao Ge, Tong Zhang, and Hongyuan Liu. 2024. ngAP: Non-blocking Large-scale Automata Processing on GPUs. In *ASPLOS*. ACM, New York, NY, USA, 268–285. doi:10.1145/3617232.3624848

[10] Torsten Hoefler and Roberto Belli. 2015. Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses When Reporting Performance Results. In *SC*. ACM, New York, NY, USA, Article 73, 12 pages. doi:10.1145/2807591.2807644

[11] Sunpyo Hong and Hyesoon Kim. 2009. An Analytical Model for a GPU Architecture with Memory-Level and Thread-Level Parallelism Awareness. In *ISCA*. ACM, New York, NY, USA, 152–163. doi:10.1145/1555754.1555775

[12] Bastian Köpcke, Sergei Gorlatch, and Michel Steuwer. 2024. Descend: A Safe GPU Systems Programming Language. *Proc. ACM Program. Lang. (PLDI)* 8, PLDI (2024), 841–864. doi:10.1145/3656411

[13] Alexis Le Glaunec, Lingkun Kong, and Konstantinos Mamouras. 2024. HybridSA: GPU Acceleration of Multi-pattern Regex Matching using Bit Parallelism. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 331 (2024), 30 pages. doi:10.1145/3689771

[14] Hongyuan Liu, Sreepathi Pai, and Adwait Jog. 2020. Why GPUs are Slow at Executing NFAs and How to Make them Faster. In *ASPLOS*. ACM, New York, NY, USA, 251–265. doi:10.1145/3373376.3378471

[15] Hongyuan Liu, Sreepathi Pai, and Adwait Jog. 2023. Asynchronous Automata Processing on GPUs. *Proc. ACM Meas. Anal. Comput. Syst. (POMACS)* 7, 1 (2023), 1–27. doi:10.1145/3579453

[16] Simon Moll and Sebastian Hack. 2018. Partial Control-Flow Linearization. In *Proc. PLDI*. ACM, New York, NY, USA, 543–556. doi:10.1145/3192366.3192413

[17] NVIDIA. 2022. Warp: A Python Framework for High-Performance GPU Simulation and Graphics. https://github.com/NVIDIA/warp.

[18] NVIDIA. 2025. cuTile and CUDA Tile IR: a tile-level model and MLIR IR, with a Tile IR backend for OpenAI Triton. CUDA 13.1. Quotations from the CUDA Programming Guide, *Writing Tile Kernels*, https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/writing-tile-kernels.html; see also https://github.com/NVIDIA/cuda-tile.

[19] NVIDIA Corporation. 2017. NVIDIA Tesla V100 GPU Architecture: The World's Most Advanced Data Center GPU. Whitepaper WP-08608-001-v1.1; introduces Independent Thread Scheduling.

[20] S. J. Pennycook, J. D. Sewall, and V. W. Lee. 2016. A Metric for Performance Portability. arXiv:1611.07409. https://arxiv.org/abs/1611.07409

[21] Burkhard Ringlein, Thomas Parnell, and Radu Stoica. 2025. GPU Performance Portability Needs Autotuning. arXiv:2505.03780. https://arxiv.org/abs/2505.03780

[22] Charitha Saumya, Kirshanthan Sundararajah, and Milind Kulkarni. 2022. DARM: Control-Flow Melding for SIMT Thread Divergence Reduction. In *Proc. CGO*. IEEE, Piscataway, NJ, USA, 1–13. doi:10.1109/CGO53902.2022.9741285

[23] Andrea Somaini, Filippo Carloni, Giovanni Agosta, Marco D. Santambrogio, and Davide Conficconi. 2025. Combining MLIR Dialects with Domain-Specific Architecture for Efficient Regular Expression Matching. In *CGO*. ACM, New York, NY, USA, 255–270. doi:10.1145/3696443.3708916

[24] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *MAPL*. ACM, New York, NY, USA, 10–19. doi:10.1145/3315508.3329973

[25] Vasily Volkov. 2016. *Understanding Latency Hiding on GPUs*. Ph. D. Dissertation. University of California, Berkeley. Tech. Rep. UCB/EECS-2016-143. https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-143.html

[26] Dewei Wang, Wei Zhu, Liyang Ling, Ettore Tiotto, Quintin Wang, Whitney Tsang, Julian Opperman, and Jacky Deng. 2025. ML-Triton, A Multi-Level Compilation and Language Extension to Triton GPU Programming. arXiv:2503.14985. https://arxiv.org/abs/2503.14985

[27] Lei Wang, Yu Cheng, Yining Shi, Zhengju Tang, Zhiwen Mo, Wenhao Xie, Lingxiao Ma, Yuqing Xia, Jilong Xue, Fan Yang, and Zhi Yang. 2025. TileLang: A Composable Tiled Programming Model for AI Systems. arXiv:2504.17577. https://arxiv.org/abs/2504.17577

[28] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Commun. ACM* 52, 4 (2009), 65–76. doi:10.1145/1498765.1498785

[29] Jaeyeon Won, Willow Ahrens, Joel S. Emer, and Saman Amarasinghe. 2026. Insum: Sparse GPU Kernels Simplified and Optimized with

Indirect Einsums. In *ASPLOS*. ACM, New York, NY, USA, 993–1006. doi:10.1145/3779212.3790176

[30] Zhenlin Wu, Tianao Ge, Jiajia Li, Xinyu Chen, and Hongyuan Liu. 2025. Advancing Matrix Operations for High-Performance and Memory-Efficient Automata Processing on GPUs. *ACM Trans. Archit. Code Optim. (TACO)* 22, 4 (2025), 1–26. doi:10.1145/3774656

[31] Xiao Zhang, Yaoyao Ding, Yang Hu, and Gennady Pekhimenko. 2025. Hexcute: A Tile-based Programming Language with Automatic Layout and Task-Mapping Synthesis. arXiv:2504.16214. https://arxiv.org/abs/2504.16214

[32] Keren Zhou, Mario Lezcano, Adam Goucher, Akhmed Rakhmati, Jeff Niu, Justin Lebar, Pawel Szczerbuk, Peter Bell, Phil Tillet, Thomas Raoux, and Zahi Moudallal. 2025. Linear Layouts: Robust Code Generation of Efficient Tensor Computation Using $\mathbb{F}_2$. arXiv:2505.23819. https://arxiv.org/abs/2505.23819